
DSC 40B - Discussion 03

Problem 1.

- a) Here is an alternative implementation of the `binary_search` function from lecture.

```
def binary_search(arr, target, start=0, end=None):
    if end is None:
        end = len(arr)
    if start >= end:
        return False
    middle = int(ceil((start + end) / 2))
    if arr[middle] == target:
        return True
    elif arr[middle] < target:
        return binary_search(arr, target, middle + 1, end)
    else:
        return binary_search(arr, target, start, middle)
```

Termination and correctness?

- b) Here is an alternative implementation of the `factorial` function

```
def factorial(n):
    if n == 0:
        return 1
    return n * (n - 1) * factorial(n - 2)
```

Termination and correctness?

Problem 2.

- a) State (but do not solve) the recurrence relation describing this function's run time.

```
import random
def foo(n):
    if n <= 2:
        return
    for i in range(n):
        for j in range(i, n):
            print(i)
    return foo(n//2) + foo(n//2)
```

- b) Suppose a binary search is performed on the following array using the implementation of `binary_search` from lecture. What is the worst case number of equality comparisons that would be made to search for an element in the array? That is, what is the greatest number of times that `arr[middle] == target` can be run?

[1, 4, 7, 8, 8, 10, 15, 51, 60, 65, 71, 72, 101]

- c) State (but do not solve) the recurrence relation describing this function's run time.

```
def fibonacci(n):
    if n == 0:
        return 0
```

```

        if n == 1:
            return 1
        return fibonacci(n - 1) + fibonacci(n - 2)

```

How can we improve the run time of this function?

Problem 3.

What does this function do? What is its complexity?

```

def f(x, n):
    if n == 0:
        return 1
    y = f(x, n//2)
    if isEven(n):
        return y * y
    else:
        return y * y * x

```

Problem 4.

Solve the following recurrence relations.

a) $T(n) = T(n-1) + n$
 $T(0)=0$

b) $T(n)=4T(n/4) + n$
 $T(1)=1$

Problem 5.

Determine the recurrence relation describing the time complexity of each of the recursive algorithms below.

a)

```
def fact(n):
    if(n <= 1)
        return 1
    else
        return n*fact(n-1)
```

b) (Note: the input `arr` to the function is a python `list` object)

```

def max_arr(arr):
    if(len(arr) == 1):
        return arr[0]
    mid = len(arr)//2
    left_max = max_arr(arr[:mid])
    right_max= max_arr(arr[mid:])
    if(left_max>right_max):
        return left_max
    else:
        return right_max

```